

MANUEL UTILISATEUR

Système de Gestion de Scolarité

Projet Algorithmique et Structures de Données

Filière Licence 2 Mathématiques-Informatique

Université Iba Der Thiam de Thiès

Enseignant : Dr Abdoulaye DIALLO

Binôme : SENE — DIONE

Année 2025–2026

1 Introduction

Ce manuel décrit l'utilisation du système de gestion de scolarité développé dans le cadre du projet LMI ASD 2026. Le système implémente trois structures de données distinctes pour stocker et manipuler des enregistrements d'étudiants : le tableau statique, le tableau dynamique de pointeurs et la liste doublement chaînée.

Chaque structure supporte les opérations d'insertion, de recherche, de suppression, de mise à jour, de tri, d'agrégations statistiques et de persistance binaire sur disque.

2 Installation et Compilation

2.1 Prérequis

Avant de compiler le projet, assurez-vous d'avoir installé :

- Compilateur GCC (version 9 ou supérieure)
- Utilitaire `make`
- Python 3 avec les bibliothèques : `pandas`, `matplotlib`, `numpy` (pour la génération des courbes)

2.2 Compilation

Depuis la racine du projet, exécutez :

```
make all
```

Cette commande compile toutes les cibles du projet. Vous pouvez aussi compiler chaque cible séparément. Attention : le nom de la cible `make` ne correspond pas toujours au nom de l'exécutable produit.

Commande	Exécutable produit	Description
<code>make main_structure</code>	<code>structure</code>	Démo des types liés par pointeurs
<code>make main_bench</code>	<code>bench</code>	Programme de benchmark
<code>make main_statique</code>	<code>main_statique</code>	Test du tableau statique
<code>make main_dynamique</code>	<code>main_dynamique</code>	Test du tableau dynamique
<code>make main_liste</code>	<code>main_liste</code>	Test de la liste chaînée
<code>make main_principal</code>	<code>main_principal</code>	Menu interactif regroupant les 3 structures
<code>make clean</code>	—	Supprime les exécutables et fichiers générés

3 Menu Interactif Principal

Le programme `main_principal` regroupe l'ensemble des fonctionnalités des trois structures dans un seul menu :

```
./main_principal
```

3.1 Options du menu principal

Option	Action
— <i>Démonstrations automatiques</i> —	
1	Démo Tableau Statique
2	Démo Tableau Dynamique
3	Démo Liste Doublement Chaînée
— <i>Menus interactifs (saisie clavier)</i> —	
4	Interface Tableau Statique
5	Interface Tableau Dynamique
6	Interface Liste Doublement Chaînée
— <i>Outils</i> —	
7	Lancer le Benchmark (redirige vers l'exécutable <code>bench</code>)
0	Quitter

Les options 1 à 3 exécutent une démonstration automatique préremplie avec des données d'exemple, sans saisie clavier. Les options 4 à 6 ouvrent un sous-menu interactif propre à chaque structure, détaillé ci-dessous.

3.2 Sous-menu : Interface Tableau Statique (option 4)

Option	Action
1	Ajouter un étudiant
2	Afficher tous les étudiants
3	Rechercher par matricule
4	Rechercher par intervalle de moyenne
5	Rechercher par préfixe du nom
6	Supprimer un étudiant
7	Trier par moyenne (tri par insertion)
8	Trier par moyenne (tri rapide)
9	Statistiques générales
0	Retour au menu principal

3.3 Sous-menu : Interface Tableau Dynamique (option 5)

Option	Action
1	Ajouter un étudiant
2	Afficher tous les étudiants
3	Rechercher par matricule
4	Rechercher par intervalle de moyenne
5	Rechercher par préfixe du nom
6	Supprimer un étudiant
7	Modifier la moyenne d'un étudiant
8	Trier par moyenne (tri par insertion)
9	Trier par moyenne (tri rapide)
10	Statistiques générales
11	Sauvegarder dans un fichier binaire
12	Charger depuis un fichier binaire
0	Retour au menu principal

3.4 Sous-menu : Interface Liste Doublement Chaînée (option 6)

Option	Action
1	Ajouter en tête
2	Ajouter en queue
3	Afficher du début vers la fin
4	Afficher de la fin vers le début
5	Rechercher par matricule
6	Rechercher par intervalle de moyenne
7	Rechercher par préfixe du nom
8	Supprimer un étudiant
9	Trier par moyenne (tri par insertion)
10	Trier par moyenne (tri à bulles)
11	Statistiques générales
12	Sauvegarder dans un fichier binaire
13	Charger depuis un fichier binaire
0	Retour au menu principal

3.5 Exemple de session (Interface Tableau Dynamique)

```
Choix : 1
Matricule : 1001
Nom : Ndiaye
Prenom : Moussa
Moyenne : 14.5
Jour Mois Annee : 5 3 2003
Etudiant ajoute.

Choix : 3
Matricule : 1001
Trouve : Ndiaye Moussa -- 14.50
```

4 Démonstration de la Modélisation des Enregistrements

Un programme séparé, `structure`, illustre la modélisation des enregistrements liés par pointeurs (`Etudiant`, `Note`, `Inscription`) exigée par le cahier des charges :

```
./structure
```

Ce programme initialise un étudiant, sa note et son inscription, affiche leurs champs en accédant aux pointeurs, puis libère la mémoire allouée dynamiquement (`prenom`, `appreciation`, `filliere`).

5 Programmes de Test par Structure

Trois programmes de test permettent de tester chaque structure indépendamment avec des données pré-définies :

5.1 Tableau statique

```
./main_statique
```

Teste toutes les opérations du tableau statique : insertion triée par matricule, recherche, suppression avec décalage, tris et agrégations statistiques.

5.2 Tableau dynamique

```
./main_dynamique
```

Teste le redimensionnement automatique (`realloc × 2`), l'insertion, la recherche par matricule, par intervalle et par préfixe, les deux algorithmes de tri et la persistance binaire.

5.3 Liste doublement chaînée

```
./main_liste
```

Teste l'insertion en tête et en queue en $O(1)$, le parcours avant et arrière, la suppression avec reliaison des noeuds, et l'affichage inverse grâce aux pointeurs `prec`.

6 Benchmark et Courbes

Le benchmark mesure les temps d'exécution de chaque opération pour les trois structures, selon différentes tailles et distributions de données.

6.1 Lancer le benchmark

Compilez puis exécutez :

```
make main_bench  
./bench
```

Le benchmark génère automatiquement le fichier `resultats_benchmark.csv` contenant les mesures.

Paramètre	Valeurs
Tailles n	100 — 1 000 — 10 000 — 100 000
Distributions	Uniforme, Gaussienne, Triée
Répétitions	10 fois par mesure
Mesures	Moyenne + écart-type en millisecondes

Remarque : l'en-tête du fichier CSV indique par erreur "(s)" (secondes) à la suite des colonnes de temps, mais les valeurs réellement stockées sont en millisecondes.

6.2 Générer les courbes

Une fois `resultats_benchmark.csv` généré, lancez successivement les scripts Python du dossier `graphs/`, depuis la racine du projet :

```
python graphs/courbe_lineaire.py
python graphs/courbe_log.py
python graphs/courbe_distribution.py
python graphs/courbe_comparaison.py
python graphs/courbe_ralentissement.py
```

Les courbes PNG générées sont placées dans le dossier `graphs/` :

- `lineaire_*.png` : temps vs n (échelle linéaire)
- `log_*.png` : temps vs n (échelle logarithmique)
- `distribution_*.png` : effet de la distribution des données
- `comparaison_*.png` : mesures empiriques vs courbes théoriques
- `histogramme_ralentissement.png` : facteur de ralentissement entre la meilleure et la pire structure, par opération

Pour inclure une courbe dans le rapport LaTeX, copiez le fichier PNG souhaité du dossier `graphs/` vers le dossier `figures/` du rapport.

7 Persistance des Données

Chaque structure supporte la sauvegarde et le rechargement des données dans un fichier binaire (`.bin`).

Depuis le menu interactif (Interface Tableau Dynamique ou Interface Liste Doublement Chaînée) :

- Sauvegarder : entrez un nom de fichier (ex : `promo2026.bin`)
- Charger : entrez le nom du fichier à recharger

Le format binaire est plus compact et plus rapide à lire que le format texte. Les champs de longueur variable (prénom, filière) sont précédés de leur longueur pour permettre la reconstruction exacte.

8 Nettoyage

Pour supprimer tous les exécutables et fichiers générés (CSV, fichiers binaires, courbes PNG) :

```
make clean
```